W H I T E P A P E R v 2 . 0

CipherVault

Decentralized Encrypted File Vault, Secure Messaging, Document Signing, and On-Chain Digital Asset Marketplace

Blockchain: BridgeStone L1 (Avalanche) | Chain ID: 777000
Native Token: VELD | Smart Contract: CipherVault.sol (ERC-721)
Issued by: StoneBridge Intelligence | Version 2.0 | 2025

CONFIDENTIAL | FOR INFORMATIONAL PURPOSES ONLY

Disclaimer

This white paper has been prepared by StoneBridge Intelligence solely for informational and educational purposes. Nothing contained in this document constitutes financial, investment, legal, or tax advice. It does not represent an offer to sell, a solicitation to purchase, or a recommendation of any security, token, or financial instrument.

The information in this white paper reflects the current state of development of CipherVault and is subject to change without prior notice. Technology, tokenomics, and roadmap details may evolve as the protocol matures.

Engaging with blockchain protocols, cryptocurrencies, and smart contracts involves substantial risks including but not limited to smart contract vulnerabilities, regulatory uncertainty across jurisdictions, market price volatility, loss of private keys, and protocol-level failures. Readers are strongly encouraged to conduct independent due diligence and seek professional advice before participating in any blockchain ecosystem.

StoneBridge Intelligence makes no representations or warranties regarding the accuracy, completeness, or fitness for purpose of any information contained herein. Past performance or projections described in this document do not guarantee future results.

Table of Contents

1. Executive Summary
2. The Problem: Why Centralized Storage and Communication Are Broken
3. Vision and Mission
4. CipherVault Solution Overview
5. Technical Architecture: Three-Layer Stack
6. Smart Contract Deep Dive
7. Encryption Engine: ECDH, AES-GCM, and Key Versioning
8. Core Product Modules
9. Tokenomics: VELD
10. Security Architecture and Audit Readiness
11. Competitive Landscape
12. Use Cases
13. Gas Optimization and EIP-1559 Integration
14. Roadmap
15. Community, Governance, and Ecosystem
16. Team: StoneBridge Intelligence
17. Conclusion

1. Executive Summary

CipherVault is a full-stack Web3 platform that solves the most fundamental unsolved problem of the digital age: how can a person store sensitive data, communicate privately, and transact digital assets without trusting any third party to protect them?

The answer is a five-module ecosystem built on the BridgeStone Layer 1 blockchain, an Avalanche-based sovereign network, where every piece of sensitive data is encrypted client-side before it ever leaves the user's device. The encryption key is derived from the user's own wallet signature. No server, no administrator, and not even StoneBridge Intelligence can access the plaintext of any file stored on CipherVault.

The platform integrates encrypted IPFS file storage with on-chain NFT ownership, ECDH-based end-to-end encrypted peer-to-peer messaging, cryptographic document signing with SHA-256 and secp256k1, a trustless NFT marketplace with dual-party escrow, and a VELD token faucet for frictionless onboarding.

CipherVault is not competing to be a faster version of Dropbox or a cheaper version of DocuSign. It is building an entirely new category: sovereign digital infrastructure where the user's private key is the only authentication credential, the only access control mechanism, and the only entity that can ever unlock their data.

Parameter	Value
Protocol Name	CipherVault
Blockchain	BridgeStone L1 (Avalanche Subnet)
Chain ID	777000
Native Token	VELD
Token Type	Native L1 Gas and Utility Token
Smart Contract Standard	ERC-721 with Custom Marketplace Logic
Solidity Version	0.8.28 (Optimizer 200 runs)
Storage Layer	IPFS (Kubo Node, Content-Addressed)
Encryption	ECDH secp256k1 + AES-GCM 256-bit
Key Derivation	Wallet signature + SHA-256 (v2), ECDH shared secret (v3)
Deployed Contract	0xfEaE1829545008221d3cac836acDA2ACd39748b6
Issuer	StoneBridge Intelligence
Version	2.0 | 2025

2. The Problem: Why Centralized Storage and Communication Are Broken

The modern internet is built on centralized architecture. Users deposit their most sensitive data, private communications, and digital assets into servers owned and controlled by corporations, governments, and institutions. This model generates four categories of systemic failure that affect billions of people daily.

2.1 The Centralized Single Point of Failure

When data is stored centrally, every user's files share the same vulnerabilities as the host institution. A single successful cyberattack can expose the private data of millions of people simultaneously. According to IBM's Cost of a Data Breach Report 2023, the average cost of a single data breach reached 4.45 million USD, with healthcare breaches averaging 10.93 million USD. More importantly, 83% of organizations surveyed had experienced more than one data breach. These numbers reflect the inevitable mathematical reality: concentrated data creates concentrated risk.

IPFS, the storage layer underlying CipherVault, distributes file content across a peer-to-peer network of nodes. There is no central server to attack because there is no central server. Each file is identified by its cryptographic hash, not its location, meaning files can be retrieved from any node that holds a copy. The attack surface is not a single target but a distributed network of thousands of independent peers.

2.2 Surveillance Capitalism and the End of Privacy

Web2 storage and messaging platforms operate on a surveillance-based business model. When a user stores a document on a major cloud provider or sends a message through a mainstream communication app, they are typically agreeing to terms that permit the platform to scan, analyze, index, and in many cases share that content with advertising networks, data brokers, and in response to government requests.

A 2023 study by the Electronic Frontier Foundation found that major technology platforms collectively receive hundreds of thousands of government data requests annually, the majority of which are complied with. The architecture of these systems makes compliance structurally easy: the platform holds the encryption keys, so decrypting any user's data is trivial.

CipherVault inverts this architecture. The encryption key is derived from the user's Ethereum wallet signature and never transmitted to any server. No CipherVault server ever sees the key. No CipherVault employee can decrypt user data. The platform has no ability to comply with a data access request because there is nothing to hand over.

2.3 Platform Censorship and Deplatforming

Centralized platforms can and regularly do unilaterally terminate user access to their own data and digital assets. In 2021 alone, major cloud providers terminated accounts affecting millions of users and terabytes of data, often without warning, due to automated policy violations, content moderation decisions, or commercial disputes.

For digital asset creators, artists, content owners, and businesses, losing platform access can mean permanently losing access to their intellectual property, their income streams, and their work. The platform is the custodian, and the custodian can revoke custody at will.

On CipherVault, ownership of a digital asset is represented by an NFT on the BridgeStone L1 blockchain. The NFT is held in the user's wallet, not in a platform account. As long as the user holds their private key, ownership is guaranteed by the blockchain's consensus mechanism, not by a platform's terms of service.

2.4 The Document Integrity Crisis

In a world of photorealistic AI image generation and sophisticated document editing tools, the question of whether a digital document is authentic and unmodified has become practically unanswerable within Web2 infrastructure. Traditional digital signatures rely on certificate authorities, centralized public key infrastructure, and third-party notarization services, all of which are expensive, slow, and themselves subject to compromise.

CipherVault's Document Signer module uses the SHA-256 hash of the file content, combined with a timestamp and the signer's Ethereum address, to produce a cryptographic signature using the secp256k1 elliptic curve. This signature can be independently verified by any party with access to the document and the signature, with zero dependency on any trusted third party. Forgery is computationally infeasible.

2.5 The Scale of the Problem

To appreciate the urgency of the infrastructure CipherVault is building, consider the following data points:

* **Data Breach Exposure:** Over 33 billion records were exposed in data breaches in 2023 alone, according to IT Governance research.
* **Surveillance Scale:** The Global Surveillance Index tracked government mass surveillance programs in over 75 countries as of 2023.
* **Platform Dependency:** Over 60% of global file storage market share is controlled by three providers: AWS, Microsoft Azure, and Google Cloud, representing a historic concentration of data custody.
* **Digital Asset Vulnerability:** An estimated 3.5 million USD worth of NFTs were stolen through phishing and platform exploits in Q1 2023 alone according to Immunefi reports.
* **Messaging Privacy Gaps:** In 2023, major messaging platforms collectively received over 300,000 government data requests, complying with the majority.

CipherVault is not solving edge-case problems for privacy enthusiasts. It is building the infrastructure that a post-Web2 internet requires.

3. Vision and Mission

> *The user's private key is the only entity in the world that should ever have access to their data. Everything else is a compromise.*

Vision: CipherVault aims to become the foundational digital sovereignty layer for individuals, businesses, and institutions globally, a permanent piece of internet infrastructure where users own their data absolutely, communicate with mathematical privacy guarantees, and transact digital assets without any intermediary touching, seeing, or controlling the exchange.

Mission: StoneBridge Intelligence builds CipherVault on four non-negotiable principles:

* **Zero-Knowledge by Default:** Every file uploaded, every message sent, and every document signed is processed entirely within the user's browser or device before any data touches the network. The ciphertext that leaves the device is mathematically useless to anyone without the wallet-derived key.
* **Cryptographic Identity Sovereignty:** There are no usernames, no passwords, and no account recovery flows that require trusting a company. The user's wallet is their identity. Losing the wallet key means losing access, but no third party can also lose it for them.
* **On-Chain Immutable Ownership:** Ownership of digital assets is recorded on the BridgeStone L1 blockchain. It is verifiable by anyone, alterable by no one, and dependent on no company remaining in business or maintaining servers.
* **Seamless User Experience:** The complexity of key derivation, IPFS content addressing, and blockchain transactions is entirely abstracted. A user interacts with CipherVault the way they interact with any file storage application, with the security guarantees of military-grade cryptographic infrastructure running silently underneath.

4. CipherVault Solution Overview

CipherVault integrates five distinct product modules into one coherent ecosystem. Each module addresses a specific dimension of the problem landscape while sharing the same cryptographic infrastructure, the same blockchain identity, and the same VELD token economy.

Module	Core Function	Key Technology
Encrypted Vault	Client-side encrypted file storage with on-chain NFT ownership proof	AES-GCM + IPFS + ERC-721
Secure Messenger	End-to-end encrypted wallet-to-wallet messaging	ECDH secp256k1 + AES-GCM
Document Signer	Tamper-proof cryptographic document signing and verification	SHA-256 + secp256k1 signature
Asset Marketplace	Trustless NFT buy/sell with optional dual-party escrow	CipherVault.sol smart contract
File Sharing	Encrypted file transfer between wallets with re-encryption	ECDH key re-wrapping v3

What makes CipherVault meaningfully different from existing solutions in each category is that the five modules are not independent products bolted together. They share a unified cryptographic identity layer (the wallet), a unified ownership layer (the ERC-721 smart contract), and a unified economic layer (VELD). A file uploaded to the vault can be sold in the marketplace, shared via secure messenger, and signed with the document signer, all using the same key material and the same on-chain identity.

5. Technical Architecture: Three-Layer Stack

CipherVault is built on three interdependent layers, each serving a distinct role in the security and functionality model. No layer can be removed without breaking the others, which is by design.

5.1 Storage Layer: IPFS and Client-Side Encryption

The storage layer is where files live. Files stored on CipherVault go through the following pipeline entirely within the user's browser before any data is transmitted:

1. A 32-byte random AES content key and 12-byte IV are generated using crypto.subtle via the browser's built-in cryptographic random number generator.
2. The file content is encrypted using AES-GCM with the content key. AES-GCM provides authenticated encryption, meaning any tampering with the ciphertext after encryption is detectable.

3. The content key itself is then encrypted using a wrap key derived from the user's wallet signature. This is the wallet-binding step: the wrap key is derived by passing the user's Ethereum wallet signature of a deterministic message through SHA-256. Only the wallet that produced that signature can reproduce the wrap key.
4. The encrypted content and the wrapped (encrypted) key are bundled into a single JSON payload and uploaded to IPFS.
5. IPFS returns a Content Identifier (CID), which is the SHA-256 hash of the content. This CID is stored on-chain as the NFT's tokenURI.

The architectural result of this pipeline is that the ciphertext stored on IPFS is useless without the wrap key. The wrap key is reproducible only by the original wallet. The CID stored on-chain is the address of the ciphertext. Reconstructing the plaintext requires the wallet, the CID, the wrapped key, and the AES parameters, none of which are ever stored together on any server.

5.2 Consensus Layer: BridgeStone L1 Blockchain

BridgeStone is a sovereign Avalanche Layer 1 subnet with the following network parameters:

Network Parameter	Value
Network Name	BridgeStone
Chain ID	777000
Native Token	VELD
Consensus Protocol	Avalanche Snowman++ (single-slot finality)
Fee Model	EIP-1559 (Type 2 transactions)
Max Fee Per Gas	30 Gwei
Priority Fee	1 Gwei
RPC Endpoint	Custom subnet RPC via Avalanche L1 architecture
Avalanche Version	Avalanche Subnet-EVM compatible
IPFS Integration	Kubo Node API (local or tunnel-proxied)
Frontend Deployment	Next.js via Vercel with Vercel KV for tunnel URL resolution

The choice of a sovereign Avalanche L1 over deploying on Ethereum mainnet or a public Layer 2 was deliberate. A sovereign subnet allows CipherVault to set its own gas price parameters, ensuring that file storage and marketplace transactions remain economically accessible. The Avalanche Snowman++ consensus provides single-slot finality, meaning an NFT mint or transfer is irreversible within seconds rather than requiring multiple block confirmations.

5.3 Communication Layer: ECDH Encrypted Messaging

The communication layer enables encrypted messaging between any two wallet addresses on CipherVault. The encryption scheme uses Elliptic Curve Diffie-Hellman on the secp256k1 curve, the same curve underlying Ethereum and Bitcoin key derivation.

The ECDH key exchange works as follows. The sender takes their own private key and the recipient's public key and computes a shared secret using elliptic curve scalar multiplication. The recipient independently performs the same operation with their private key and the sender's public key. Both parties arrive at the same shared secret without either party ever transmitting their private key, and without any server ever seeing the shared secret.

The shared secret is passed through SHA-256 to derive a 256-bit AES key. The message is encrypted using AES-GCM with this derived key and a randomly generated 12-byte initialization vector. The encrypted payload stored off-chain contains the ciphertext in base64, the IV in hex, and the sender's public key. No server can decrypt this payload because no server ever has the private key of either party.

6. Smart Contract Deep Dive

The CipherVault.sol smart contract is the single source of on-chain truth for the platform. It inherits from OpenZeppelin's ERC721URIStorage (providing standard NFT functionality with metadata storage) and Ownable (providing administrative controls). The contract is deployed and immutable at address 0xfEaE1829545008221d3cac836acDA2ACd39748b6 on BridgeStone L1.

6.1 The VaultItem Data Structure

Every digital asset minted on CipherVault is represented as a VaultItem struct stored in a mapping from token ID to VaultItem. This struct contains the following fields:

* **id and tokenId:** The unique identifier for the asset on-chain.
* **seller and owner:** Payable addresses tracking custody across states.
* **price:** Listing price in VELD (wei denomination).
* **isListed:** Boolean indicating active marketplace availability.
* **isCopy:** Boolean distinguishing original mints from sendCopy operations.
* **encryptedCid:** The IPFS CID of the encrypted file payload. This is the most security-critical field; it represents the on-chain pointer to the encrypted content.
* **previewURI:** An optional unencrypted preview or thumbnail CID for marketplace display without exposing the encrypted content.
* **name and description:** Human-readable metadata for the asset.
* **useEscrow:** Boolean enabling dual-confirmation escrow for high-value trades.
* **buyer, sellerConfirmed, buyerConfirmed, isEscrowActive:** Escrow state management fields used during two-party trade resolution.

6.2 Contract Function Reference

Function	Access	Gas Limit	Description
mintToVault(tokenURI, name)	Public	400,000	Mints a new ERC-721 NFT. The tokenURI is the encrypted IPFS CID. Emits VaultItemCreated.
listAsset(tokenId, price, desc, previewURI, useEscrow)	Owner only	300,000	Lists an asset for sale. Transfers NFT custody to the contract. Optionally activates escrow mode.
updateListing(tokenId, price, desc, useEscrow)	Seller only	150,000	Updates price, description, or escrow setting without delisting.
cancelListing(tokenId)	Seller only	150,000	Removes listing and returns NFT custody to seller. Clears all escrow state.
buyAsset(tokenId)	Public (payable)	300,000	Purchases a listed asset. Standard: immediate settlement. Escrow: funds locked pending

Function	Access	Gas Limit	Description
			confirmations.
confirmTrade(tokenId)	Buyer or Seller	300,000	Dual-confirmation escrow release. Both parties must call. Atomic settlement on second confirmation.
cancelTrade(tokenId)	Buyer or Seller	150,000	Cancels active escrow. Full VELD refund to buyer. NFT returned to seller. No slippage.
transferAsset(tokenId, to, newEncryptedCid)	Owner only	150,000	Transfers NFT and atomically updates the on-chain CID. Prevents state inconsistency between ownership and encryption.
sendCopy(to, name, encryptedCid)	Public	300,000	Mints a copy of a document to another wallet, encrypted specifically for the recipient.
updateEncryptedCid(tokenId, newCid)	Owner or Seller (escrow)	150,000	Re-encrypts and updates the CID post-transfer or during escrow. Dual-context access control.
burnAsset(tokenId)	Owner only	150,000	Permanently destroys the NFT. Irreversible. Emits AssetBurned.
getMyAssets()	Public view	No gas (read)	Returns all VaultItems where caller is owner, seller, or escrow buyer.
getAllListedAssets()	Public view	No gas (read)	Returns all active marketplace listings (isListed = true, isEscrowActive = false).

6.3 On-Chain Event System

Every state-changing operation emits a blockchain event, enabling the frontend to update in real time without polling the chain. The complete event set is:

* **VaultItemCreated(tokenId, owner):** Emitted on every successful mint or copy.
* **AssetListed(tokenId, price):** Emitted when an asset enters the marketplace.
* **ListingUpdated(tokenId, price):** Emitted on price or escrow mode changes.
* **ListingCancelled(tokenId):** Emitted when a seller removes a listing.
* **AssetSold(tokenId, buyer, price):** Emitted on successful direct purchase or escrow completion.
* **AssetTransferred(tokenId, from, to):** Emitted on peer-to-peer transfers.
* **EscrowUpdate(tokenId, status):** Emitted with status 'Start', 'Confirm', or 'Cancel' for escrow state tracking.
* **AssetBurned(tokenId):** Emitted on permanent NFT destruction.

7. Encryption Engine: ECDH, AES-GCM, and Key Versioning

The encryption engine (crypto-engine.ts) is one of the most technically sophisticated components of CipherVault. It manages three encryption versions with backward compatibility, a wallet-bound key caching system, and ECDH-based key re-encryption for ownership transfers and marketplace trades.

7.1 Encryption Version History

* **Version 1 (Legacy):** AES key stored in plaintext alongside the encrypted file in the IPFS payload. This version is retained for backward compatibility only. It provides file confidentiality from external parties but not from the IPFS node operator. New uploads never use v1.
* **Version 2 (Wallet-Bound, Current Default):** The AES content key is encrypted using a wrap key derived from the user's wallet signature. The wrap key is deterministically derived by signing a network-specific constant message with the wallet's private key, passing the resulting signature bytes through SHA-256, and importing the result as a CryptoKey with AES-GCM semantics. The wrap key never leaves the browser's memory. Only the encrypted content key and the ciphertext are sent to IPFS.
* **Version 3 (ECDH Re-encryption, Used on Transfer):** When an asset changes ownership through marketplace purchase or direct transfer, the new owner cannot decrypt the file with the original seller's wrap key. Version 3 solves this atomically. The seller decrypts the AES content key using their v2 wrap key, then re-encrypts it using a shared ECDH secret derived from the seller's private key and the buyer's public key. The resulting v3 payload is uploaded to IPFS, and the new CID is written on-chain in the same transaction as the NFT transfer. This atomic operation ensures there is no window in which the ownership record and the decryption capability are out of sync.

7.2 Wrap Key Caching Architecture

Deriving the wrap key requires the user to sign a message with their wallet, which in browser wallet implementations (MetaMask, Rabby, etc.) typically produces a popup confirmation. To avoid requesting this signature on every file operation, CipherVault implements a session-level wrap key cache.

The cache is an in-memory Map keyed by wallet address. The wrap key is derived once when the user explicitly opens their vault (via the unlockVaultKey function). It is stored in the cache for the duration of the browser session. On logout, the clearVaultKey function removes the key from the cache. The key is never written to localStorage, sessionStorage, or any persistent browser storage.

This design means the user sees exactly one wallet signature request per session for encryption operations, regardless of how many files they upload or download.

7.3 Secure Messaging Encryption

The message encryption module (message-crypto.ts) implements a complete ECDH messaging protocol using the browser's Web Crypto API. For each message, the following steps occur:

1. The sender's wallet signing key computes a shared secret using computeSharedSecret with the recipient's public key.
2. The shared secret bytes are hashed with SHA-256 using crypto.subtle.digest to produce a 256-bit derived key material.
3. The derived key material is imported as a non-extractable AES-GCM key using crypto.subtle.importKey.
4. A fresh 12-byte IV is generated using crypto.getRandomValues for each message.
5. The plaintext is encoded using TextEncoder and encrypted using AES-GCM.
6. The encrypted payload contains the base64-encoded ciphertext, the hex IV, and the sender's public key for recipient-side decryption.

Decryption is the symmetric reverse: the recipient's wallet computes the same shared secret using their private key and the sender's public key field from the payload, derives the same AES key, and decrypts. The security of ECDH means that any third party intercepting the payload cannot reproduce the shared secret without possessing at least one of the two private keys.

8. Core Product Modules

8.1 Encrypted Vault: Sovereign File Storage

The Encrypted Vault is the foundational module of CipherVault. Users can upload files of any format and size into their personal vault. Every file is encrypted at the v2 wrap-key level before leaving the browser. The resulting ciphertext is pushed to IPFS, the CID is minted as an NFT, and the user's vault view is populated from their on-chain asset registry via getMyAssets.

The vault supports the full lifecycle of a digital asset: upload, view, download and decrypt, list for sale, transfer to another wallet, share as a copy, sign as a document, and permanently burn. Each operation has a corresponding smart contract function and emits a real-time event that the frontend subscribes to.

Activity is tracked in a persistent activity ledger (activity-store.ts) using Zustand state management with local persistence. The ledger retains up to 200 events per wallet address, with metadata including transaction hash, token ID, counterparty address, timestamp, and VELD amount. Activity types span the full range of user operations: upload, list, delist, buy, sell, transfer in, transfer out, escrow start, escrow confirm, escrow cancel, message sent, message received, sign, burn, and faucet.

8.2 Secure Messenger: Zero-Knowledge P2P Communication

The Secure Messenger enables encrypted communication between any two wallet addresses. There is no message server. There is no message database on CipherVault's infrastructure. Encrypted message payloads are stored in a decentralized manner, and the platform maintains zero knowledge of message content.

The contact system (contact-store.ts) maintains a local, persistent address book with name, Ethereum address, optional notes, and an emoji identifier. Contact search is available by name, address, or note content. The contact book is stored locally using Zustand with persistence and never transmitted to any server.

Notifications for incoming messages are managed by a real-time notification system (notif-store.ts) that maintains a priority queue of up to 50 notifications with types covering trade updates, messages, warnings, and system events. Notifications include optional deep-link actions to navigate the user to the relevant section of the application.

8.3 Document Signer: Cryptographic Notarization

The Document Signer provides mathematically verifiable proof that a specific document existed in a specific state at a specific time, and was signed by a specific Ethereum address. The signing process:

1. Computes the SHA-256 hash of the file's binary content using crypto.subtle.digest.
2. Constructs a structured message string: 'CipherVault Document Signature' followed by the hash and the Unix timestamp in milliseconds.
3. Signs the structured message using the user's Ethereum wallet via wallet.signMessage (which applies the standard Ethereum message prefix before signing).

4. Returns the content hash, signature bytes, signer address, and timestamp as the complete proof record.

Verification is performed by any party who holds the original document and the proof record. The verifier recomputes the SHA-256 hash of their copy of the document, reconstructs the identical structured message, and calls ethers.verifyMessage with the reconstructed message and the provided signature. If the recovered address matches the claimed signer address, the document is confirmed as unmodified and the signature is confirmed as genuine. Any modification to even a single byte of the document produces a completely different SHA-256 hash, causing verification to fail.

This mechanism is suitable for legally binding digital signatures, intellectual property registration, non-disclosure agreements, and any context where tamper-evident document provenance is required.

8.4 Asset Marketplace: Trustless Digital Commerce

The marketplace module enables peer-to-peer buying and selling of digital assets without any platform intermediary controlling funds or the exchange process. The complete transaction flow for a standard (non-escrow) sale:

1. Seller calls listAsset with a VELD price. The NFT is transferred to the contract's custody.
2. Buyer calls buyAsset with exactly the listed VELD price in the transaction value. Any discrepancy causes the transaction to revert.
3. The contract immediately transfers the NFT to the buyer's address and transfers the VELD payment to the seller's address. Both transfers are atomic in a single transaction.
4. The seller re-encrypts the file using the v3 ECDH scheme for the buyer's wallet and calls updateEncryptedCid with the new CID.

For high-value trades where both parties want additional assurance, the escrow flow provides a dual-confirmation mechanism:

1. Seller lists the asset with useEscrow = true.
2. Buyer calls buyAsset. Funds are locked in the contract and isEscrowActive is set to true. The asset is removed from the general marketplace listing.
3. During the escrow period, the seller re-encrypts the file for the buyer and updates the encrypted CID on-chain.
4. Both buyer and seller independently call confirmTrade. On the second confirmation, the contract atomically transfers the NFT to the buyer and releases the VELD to the seller.
5. Either party can call cancelTrade at any point before double-confirmation, which refunds the full VELD amount to the buyer and returns the NFT to the seller with no deductions.

8.5 File Sharing and Secure Transfer

CipherVault supports two distinct models for sharing encrypted files with other wallet addresses.

The sendCopy model mints an entirely new NFT to the recipient's wallet, encrypted at v3 using the sender's private key and the recipient's public key. The original file remains in the sender's vault unchanged. The recipient receives a fully independent, wallet-bound copy of the file that they can decrypt, manage, and even resell as their own asset.

The transferAsset model moves ownership of the original NFT. This function takes the recipient's address and an optional new encrypted CID. If a new CID is provided, the contract updates the on-

chain tokenURI atomically in the same transaction as the ownership transfer. This atomic re-encryption-and-transfer is a critical security property: it eliminates the window between an ownership transfer and a key update during which a new owner might be unable to access their asset, or during which the old owner retains decryption capability.

9. Tokenomics: VELD

VELD is the native Layer 1 coin of the BridgeStone blockchain. It is used for gas fees on every transaction, as the settlement currency in the CipherVault marketplace, and as the denomination for escrow amounts held by the smart contract.

9.1 Token Parameters

Parameter	Value
Token Name	VELD Token
Token Symbol	VELD
Token Type	Native L1 Coin (EIP-1559)
Network	BridgeStone L1 | Chain ID 777000
Supply Model	Fixed (Static, Non-Inflationary)
Fee Mechanism	EIP-1559 Base Fee + Priority Fee (Tip)
Max Fee Per Gas	30 Gwei (protocol-configured override)
Max Priority Fee	1 Gwei
Faucet	Automated VELD distribution for new wallet onboarding

9.2 Fixed Supply Rationale

CipherVault adopts a fixed supply model for VELD. This decision is based on three principles:

* **Scarcity and Value Preservation:** A non-inflationary supply guarantees that existing VELD holders are not diluted by new issuance. As the platform's utility grows and demand for VELD increases with marketplace activity and gas consumption, the fixed supply ensures that value accretes to existing holders rather than being absorbed by new token creation.
* **Predictable Economics:** Fixed supply removes a class of monetary risk from marketplace participants. A seller listing a digital asset for 100 VELD can be confident that the value of 100 VELD is not undermined by protocol-level inflation between listing and settlement.
* **Alignment with EIP-1559:** The EIP-1559 fee mechanism, which burns the base fee portion of every gas transaction, creates mild deflationary pressure on circulating supply as network usage grows. This mechanical deflation compounds the value of fixed supply over time.

9.3 VELD Utility

* **Gas Fees:** Every on-chain transaction on BridgeStone L1 requires VELD for gas. Operations include NFT minting, marketplace listing and purchase, escrow confirmation and cancellation, file transfer, and burning.
* **Marketplace Settlement:** All CipherVault marketplace purchases are denominated and settled in VELD. The escrow contract holds VELD during dual-confirmation trades.
* **Faucet Access:** New users can request an initial VELD allocation through the built-in faucet API (/api/faucet), enabling frictionless onboarding without requiring an external exchange purchase to start using the platform.

9.4 Gas Cost Reference by Operation

Operation	Gas Limit	Actual Usage	Estimated VELD Cost (at 30 Gwei)
mintAsset	400,000	150,000 to 300,000	0.004500 to 0.009000 VELD
listAsset	300,000	150,000 to 250,000	0.003750 to 0.007500 VELD
buyAsset	300,000	120,000 to 250,000	0.003600 to 0.007500 VELD
confirmTrade	300,000	150,000 to 250,000	0.004500 to 0.007500 VELD
transferAsset	150,000	60,000 to 100,000	0.001800 to 0.003000 VELD
sendCopy	300,000	120,000 to 200,000	0.003600 to 0.006000 VELD
updateEncryptedCid	150,000	80,000	0.002400 VELD
cancelListing	150,000	60,000	0.001800 VELD
cancelTrade	150,000	80,000 to 120,000	0.002400 to 0.003600 VELD
burnAsset	150,000	80,000 to 120,000	0.002400 to 0.003600 VELD

10. Security Architecture and Audit Readiness

CipherVault's security model is built on the principle that security failures should be cryptographically impossible, not merely administratively prevented. The following describes the specific security properties of each layer.

10.1 Client-Side Encryption Guarantees

* **Zero Server Knowledge:** No plaintext data ever reaches CipherVault's servers or IPFS nodes. The encryption pipeline completes entirely within the browser's Web Crypto API before any network request is made.
* **Non-Extractable Keys:** AES keys are imported using importKey with extractable: false, meaning the JavaScript runtime cannot serialize the key material back to raw bytes after import. Even a compromised frontend script cannot exfiltrate the session key.
* **Authenticated Encryption:** AES-GCM provides both confidentiality and integrity. Any bit-flip or tampering with the ciphertext after encryption causes the decryption operation to throw a DOMException, making undetected tampering impossible.
* **Fresh Randomness:** Every upload operation generates a fresh 32-byte content key and 12-byte IV using crypto.getRandomValues. Keys are never reused across files.

10.2 Smart Contract Security

* **OpenZeppelin Audited Base:** CipherVault.sol inherits from OpenZeppelin's battle-tested ERC721URIStorage and Ownable contracts. The OpenZeppelin library is the most widely audited smart contract library in existence.
* **Atomic Re-encryption and Transfer:** The transferAsset function updates the encrypted CID and transfers the NFT in a single atomic transaction. There is no intermediate state in which ownership has changed but the encryption has not. This prevents the exploit scenario where a new owner cannot access their asset or an old owner retains access.
* **Access Control on CID Updates:** The updateEncryptedCid function implements dual-context access control: during active escrow, only the seller can update the CID (enabling them to re-encrypt for the buyer); outside escrow, only the current owner can update. This prevents unauthorized key rotation attacks.
* **Price Validation:** The buyAsset function uses require(msg.value == price) to ensure exact payment. Over-payment and underpayment both cause the transaction to revert.
* **Escrow State Machine:** The escrow lifecycle follows a strict state machine. Transitions are: Listed to EscrowActive (via buyAsset), EscrowActive to Sold (via dual confirmTrade), EscrowActive to Cancelled (via cancelTrade). Invalid state transitions revert.

10.3 Gas Optimization and Fee Safety

CipherVault implements a centralized gas configuration system (gas-config.ts) that overrides ethers.js's default fee data fetching behavior. Without this override, ethers.js queries the network for fee estimates, which can return values significantly higher than configured limits on some subnet configurations.

The patchProviderFeeData function replaces the provider's getFeeData method with a function that always returns the protocol-configured values (30 Gwei max fee, 1 Gwei priority fee). This ensures predictable transaction costs regardless of network congestion patterns on the BridgeStone subnet.

10.4 Audit Roadmap

CipherVault is preparing for a formal third-party smart contract security audit covering the following scope:

* **CipherVault.sol:** Full function-by-function review including reentrancy analysis, access control verification, arithmetic overflow checks, event emission completeness, and escrow state machine verification.
* **Encryption Implementation:** Review of the crypto-engine.ts key derivation, wrapping, and re-encryption logic against published standards for AES-GCM and ECDH on secp256k1.
* **API Security:** Review of server-side proxy routes for IPFS, RPC, faucet, and public key storage for injection vulnerabilities and access control gaps.

Results of security audits will be published publicly and in full. Any findings and remediation steps will be disclosed transparently to the community regardless of their severity.

11. Competitive Landscape

CipherVault operates at the intersection of several established market categories. Understanding how CipherVault differentiates from existing players in each category is essential for appreciating the unique value proposition.

Platform	Category	Key Limitation vs CipherVault
Dropbox / Google Drive	Centralized cloud storage	Server holds encryption keys. Provider can access, share, or lose user data. Subject to government requests.
ProtonDrive / Tresorit	Encrypted cloud storage	Centralized server infrastructure. No on-chain ownership. No NFT transfer or marketplace.
IPFS (Bare)	Decentralized storage	No encryption layer. No key management. No on-chain ownership. Raw content-addressing only.
Filecoin / Arweave	Decentralized storage with incentives	No integrated encryption. No marketplace. No messaging. Storage incentives only.
Signal / Telegram (MTProto)	Encrypted messaging	Centralized servers. Identity tied to phone number. No wallet identity. No file ownership layer.
DocuSign / Adobe Sign	Digital signature	Certificate authority dependency. Centralized account. Expensive. No blockchain proof.
OpenSea / Blur	NFT	No encryption layer. Metadata is typically public. No secure

Platform	Category	Key Limitation vs CipherVault
	marketplace	messaging. No document signing.
LooksRare / Zora	NFT marketplace	Same as OpenSea. Web2 reliance for metadata. No privacy primitives.
Lit Protocol	Decentralized key management	External dependency for key management. Complex integration. No integrated storage or messaging.
CipherVault	All of the above, integrated	Client-side encryption + IPFS + ERC-721 ownership + ECDH messaging + document signing + escrow marketplace. One platform, one key.

CipherVault's differentiating insight is that privacy, ownership, communication, and commerce are not four separate problems requiring four separate platforms. They are four dimensions of a single problem: digital sovereignty. The same wallet key that derives the file encryption wrap key also signs the marketplace transaction, also generates the ECDH messaging secret, and also signs the document notarization. This unification of cryptographic identity across all product dimensions is CipherVault's core architectural innovation.

12. Use Cases

12.1 Individual Privacy and Sovereign Data Storage

Any individual who wants to store sensitive personal documents, private communications, or valuable digital files without trusting a cloud provider is a CipherVault user. Medical records, financial documents, identity papers, private photographs, legal contracts, and personal communications can all be stored in the CipherVault vault with the mathematical guarantee that no third party can access them without the user's private key.

12.2 Investigative Journalism and Whistleblowing

Journalists working on sensitive investigations need to store evidence, communicate with sources, and prove document authenticity without creating a centralized target for adversaries. CipherVault provides all three: encrypted storage that cannot be subpoenaed by accessing a server, ECDH messaging that produces no cleartext on any server, and cryptographic document signing that proves a document existed in a specific form at a specific time.

12.3 Healthcare and Electronic Health Records

Medical institutions that need to share patient records with patients, with other providers, or with researchers can use CipherVault to ensure that records are encrypted specifically for the recipient's wallet and cannot be accessed by any other party. The sendCopy mechanism allows a clinic to mint an encrypted copy of a patient's records to the patient's wallet, where only the patient can decrypt them. The original remains in the clinic's vault.

12.4 Legal and Corporate Document Management

Law firms, accounting firms, and corporate legal departments handling non-disclosure agreements, merger documents, financial projections, and trade secret materials can use CipherVault's vault and document signer as a zero-trust document management system. The escrow mechanism supports deal structures where document access is released only upon confirmation by both parties.

12.5 Creative Economy and Intellectual Property

Artists, musicians, writers, and software developers can use CipherVault to register their creative works as NFTs with cryptographic timestamps, establishing prior art and ownership before publication. Works can be encrypted and sold through the marketplace: the buyer receives a v3 re-encrypted copy that only their wallet can decrypt, providing trustless digital rights management without licensing intermediaries or DRM servers.

12.6 Enterprise Secure Collaboration

Enterprises managing classified or commercially sensitive data can deploy CipherVault as an internal vault system where access is governed by cryptographic wallet identity. The contact system and messaging module enable secure team communication without a central messaging

server that can be compromised. The activity ledger provides an auditable, immutable record of all file access and transfer events.

13. Gas Optimization and EIP-1559 Integration

One of the practical barriers to mainstream blockchain adoption is unpredictable and often prohibitive transaction costs. CipherVault's BridgeStone L1 deployment, combined with a centralized gas configuration architecture, solves this problem at the infrastructure level.

13.1 EIP-1559 on BridgeStone L1

BridgeStone L1 uses EIP-1559 transaction type 2 (0x02) exclusively. This means every transaction specifies both a maxFeePerGas (the absolute maximum the user will pay per unit of gas) and a maxPriorityFeePerGas (the tip to the block producer). The actual fee is computed as min(maxFeePerGas, baseFee + maxPriorityFeePerGas).

CipherVault configures maxFeePerGas at 30 Gwei and maxPriorityFeePerGas at 1 Gwei across all transaction types. The patchProviderFeeData function in gas-config.ts injects these values directly into the ethers.js provider, overriding the default behavior of querying the network for fee estimates.

13.2 Per-Operation Gas Limit Strategy

Rather than using a single uniform gas limit for all operations, CipherVault configures operation-specific gas limits in the GAS_LIMITS object in gas-config.ts. Gas limits are set conservatively above actual observed usage to prevent out-of-gas failures, while being tight enough to prevent significant overpayment.

The gasOverride function constructs a complete override object (maxFeePerGas, maxPriorityFeePerGas, gasLimit) for a given operation. The gasOverrideWithValue function adds a VELD value field for payable operations such as buyAsset. The estimateCost function computes the worst-case VELD cost for any operation at the configured gas price.

13.3 RPC and IPFS Proxy Architecture

All browser-to-blockchain and browser-to-IPFS communication is routed through same-origin API routes (/api/rpc and /api/ipfs) hosted on the Next.js frontend. This proxy architecture serves two purposes: it eliminates CORS restrictions that would prevent direct browser connections to the blockchain node and IPFS API, and it allows the backend URL to be dynamically resolved from Vercel KV storage without hardcoding or requiring a frontend redeployment when the tunnel URL changes.

Tunnel URL management is handled by tunnel-url.ts, which reads the active RPC and IPFS URLs from Vercel KV as the primary source of truth. When a tunnel restarts (during development with ngrok, or when a production endpoint changes), only the KV value needs to be updated. All API routes automatically pick up the new URL on their next invocation.

14. Roadmap

Phase 1: Foundation (Completed)

* **BridgeStone L1 deployment with Chain ID 777000 and VELD native token.**
* **CipherVault.sol v2.1 deployed with all marketplace, escrow, and transfer functions.**
* **IPFS integration with Kubo node and same-origin proxy routing.**
* **Encryption engine v2 (wallet-bound AES wrap key) fully operational.**
* **Encryption engine v3 (ECDH buyer re-encryption on confirmTrade) fully operational.**
* **Activity ledger, notification system, and contact book fully implemented.**
* **EIP-1559 gas override system with per-operation gas limit configuration.**
* **Vercel KV tunnel URL resolution for production RPC and IPFS endpoint management.**
* **VELD faucet for new wallet onboarding.**

Phase 2: Security and Scale (Active)

* **Third-party smart contract security audit with full public disclosure.**
* **Formal verification of CipherVault.sol core invariants.**
* **Bug bounty program launch with VELD-denominated rewards.**
* **Mobile-optimized progressive web app for vault access on mobile devices.**
* **Expanded gas limit telemetry and on-chain gas usage analytics dashboard.**
* **Advanced marketplace features: auction support, bundle listings, royalty configurations.**
* **Public testnet deployment with documentation and developer onboarding.**

Phase 3: Ecosystem Expansion

* **DAO governance integration with on-chain proposal and voting contracts.**
* **Multi-signature vault support for team and institutional use cases.**
* **Cross-chain NFT bridge enabling VELD and CipherVault NFTs to interoperate with Ethereum and other EVM networks.**
* **Enterprise API: REST and SDK interfaces for institutional integrators.**
* **Programmable time-locked access: share files with automatic access expiration.**
* **Decentralized IPFS pinning incentives: VELD rewards for nodes that pin CipherVault content.**

Phase 4: Protocol Maturity

* **Quantum-resistant cryptographic signatures: migration planning for post-quantum key derivation.**

*** Zero-knowledge proof integration for privacy-preserving ownership verification.**

*** SLA guarantees for enterprise vault deployments.**

*** Regulatory compliance architecture for GDPR, HIPAA-aligned deployments.**

*** HAVEN protocol integration: CipherVault secure vault as evidence storage layer for humanitarian impact proofs.**

15. Community, Governance, and Ecosystem

15.1 Open Development Philosophy

CipherVault is developed with a commitment to technical transparency. The smart contract source code is publicly available and verifiable. The deployed bytecode on BridgeStone L1 can be independently verified against the published source. All protocol architecture decisions, security audit findings, and major development milestones are disclosed publicly.

15.2 Governance Roadmap

CipherVault will transition to DAO-based governance in Phase 3 of the roadmap. The governance model is designed around the principle that control of the protocol should be proportional to stake in the ecosystem's success, not proportional to token wealth alone. Governance proposals will cover protocol parameter changes, treasury allocation, new feature prioritization, and security response procedures.

15.3 Developer Community

* **Documentation:** Comprehensive technical documentation covering the smart contract ABI, encryption engine API, gas configuration system, and IPFS integration patterns.
* **Bug Bounty:** A VELD-denominated bug bounty program with rewards scaled to vulnerability severity. Critical findings affecting user fund safety receive maximum bounties.
* **Hackathons:** Periodic hackathons with VELD prizes for projects building on CipherVault's open components, particularly around the encryption engine, document signing module, and marketplace smart contract.
* **Integration Grants:** VELD grants for developers and teams building integrations that bring CipherVault's privacy primitives to new use cases or user populations.

15.4 Institutional and Partnership Ecosystem

* **NGO Integration:** Healthcare organizations, legal aid providers, and human rights NGOs requiring secure document management and private communication infrastructure.
* **CSR Integration:** Corporations with corporate social responsibility mandates can use the CipherVault vault as a verifiable document registry for impact reporting.
* **HAVEN Protocol Synergy:** CipherVault and HAVEN (StoneBridge Intelligence's humanitarian incentive protocol) share the same blockchain infrastructure. CipherVault's secure vault is a natural evidence storage layer for HAVEN's proof-of-beneficial-action verification pipeline.

16. Team: StoneBridge Intelligence

CipherVault is designed, developed, and operated by StoneBridge Intelligence, a technology organization building foundational infrastructure at the intersection of cryptography, decentralized systems, and practical privacy engineering.

StoneBridge Intelligence operates on the conviction that privacy-preserving technology should not be the exclusive domain of well-funded institutions or technical experts. CipherVault is the concrete implementation of this conviction: production-grade, cryptographically rigorous, and designed to be usable by anyone with a Web3 wallet.

The team brings depth across five domains that are all required to build CipherVault correctly:

* **Smart Contract Engineering:** Solidity 0.8.x development, OpenZeppelin integration, ERC-721 customization, and EVM gas optimization on Avalanche L1 subnets.
* **Cryptographic Engineering:** Applied implementation of ECDH key exchange, AES-GCM authenticated encryption, SHA-256 hashing, and secp256k1 digital signatures using the W3C Web Crypto API and the ethers.js cryptographic primitives.
* **Distributed Systems:** IPFS content addressing, Kubo node operation, content routing, and decentralized storage persistence strategies.
* **Frontend Engineering:** Next.js application architecture, Zustand state management with persistence, ethers.js blockchain integration, and progressive web app design for cross-device compatibility.
* **Protocol Security:** Threat modeling for key management systems, smart contract access control, and cryptographic protocol design.

17. Conclusion

> *Sovereignty is not a feature. It is the precondition for all other features.*

Every design decision in CipherVault traces back to a single architectural commitment: the user's private key is the ultimate authority over their data, their communications, their digital assets, and their identity. Everything that does not support this commitment has been designed out of the system.

The problem CipherVault solves is not a niche technical problem. It is the central infrastructure gap of the digital age. Billions of people daily entrust their most sensitive data and most valuable digital assets to corporations that are structurally incentivized to monetize, surveil, and potentially betray that trust. The technical tools to eliminate this dependency have existed for years. What has been missing is an integrated platform that assembles them into a product that ordinary people can use.

CipherVault is that platform. It is built on cryptographic primitives that have been mathematically validated. It is deployed on a blockchain that provides single-slot transaction finality and immutable ownership records. It is engineered with the transparency of open-source development and the rigor of institutional-grade security architecture.

The BridgeStone L1 blockchain, the VELD token economy, the ECDH encryption engine, the ERC-721 ownership registry, and the five integrated product modules collectively form a coherent system where the security guarantees are not dependent on trusting any person, company, or server. They are dependent on mathematics and on the economic security of the blockchain consensus. These are guarantees that hold at scale, under adversarial conditions, and permanently.

CipherVault is not a vision. It is deployed. The smart contract address is public. The code is auditable. The encryption engine is implementable by anyone who reads this document.

The question is not whether sovereign digital infrastructure is possible. CipherVault proves that it is. The question is how quickly the world adopts it.

CipherVault | StoneBridge Intelligence | BridgeStone L1 | VELD

Private by Design. Sovereign by Default. Open to All.